

COMP101

Introduction to Programming

2025-26

Lecture-17_2

Iteration Python – ‘for in range’

for loop: syntax - range

range() has two forms:

- single integer as a parameter

```
for <COUNTER> in range (max_value) :  
    <STATEMENTS>
```

- two integers as parameters

```
for <COUNTER> in range (min_value , max_value) :  
    <STATEMENTS>
```

- <counter> is also called the 'range variable'
- Sequence generated is of form:
 - min_value, min_value+1, . . . , max_value-1

for loop: range() function

- Generates a sequence of numbers which can be iterated over by a for loop
- single integer as a parameter
 - `range(max_value)`
 - The starting value default is 0
 - Can take one parameter unlike for-in construct which would give an error
- two integers as parameters
 - `range (min_value, max_value)`
 - `min_value` overrides default start value of 0

For loop: range (max_value)

for i in range (max_value)

- max_value is always excluded from the output

for i in range (6): generates sequence 0,1,2,3,4,5
6 numbers in the sequence
range() start default is 0

for i in range(3): generates a sequence 0, 1, 2
3 numbers in the sequence

for i in (3): gives an error
‘in’ without ‘range’ only works with
comma-separated sequences, e.g. (0,3)

for loop: range (max_value)

```
for i in range (10):  
    print ("Hello")
```

Code does not need to o/p the counter.
'Hello' outputs ten times down the screen
If you want it across the screen use:
`print ("Hello",end=" ")`

```
for i in range(10):  
    print ("Hello", i)
```

o/p the string 'Hello' followed by the counter;
Counter default start is 0;
'Hello' outputs ten times down the screen
with the counter after it
Hello 0
Hello 1
...
Hello 9

Historically, counter variables use i, j or k – go alphabetically if need more.
i means 'iterator'. It is a temporary var holding the integer value of the current position in the loop – always starts at 0 unless over-ridden.
Can initialise the counter (usually to zero) before entering the loop

for loop: range (min_value, max_value)

```
for i in range (10):  
    print ("Hello")
```

- generates a sequence 0 to 9
 - loop counter (i) always starts at 0
 - Loop 10 times "Hello" outputs ten times
 - 'start at zero (default) and stop at the max_value-1'

```
for i in range(0,10):    #min_value = 0, max_value = 10  
    print ("Hello")
```

- This generates the same sequence as above
 - but uses (min_value, max_value)
 - Allows us to say where we want the sequence to start and stop (we can over-ride the default zero start)

for loop: range (min_value, max_value)

for i in range (0,5) generates sequence 0,1,2,3,4

#same as coding for i in range(5)

for i in range (1,5) generates sequence 1,2,3,4

for i in range (1,6) generates sequence 1,2,3,4,5

for i in range(3,6): generates sequence 3, 4, 5

for i in range (3, 20, 2): #last parameter is the increment

generates sequence 3 5 7 9 11 13 15 17 19

- Sequence starts at 0 unless over-ridden and stops when the max-value is read in
- max_value is the 'sentinel' - always excluded from output

for loop: range() min_value vs max_value

```
for i in range(5):
```

```
    print (i)
```

0

1

2

3

4

o/p is correct - five outputs;
'range' default start is 0,
so final o/p is max_value-1
max_value never outputs

```
for i in range(1,5):
```

```
    print (i)
```

1

2

3

4

o/p is correct:
We have over-ridden the start default of 0 by starting it at 1;
max_value never outputs;
If we want o/p of 1,2,3,4,5 change the range to `in range(1,6):`

For loop: range() using a var

```
maxNum = int(input("Enter the stop number: ")) 8
```

```
for i in range (1, maxNum):
```

```
    print (i,"",end="")    #o/p on one line
```

```
1 2 3 4 5 6 7
```

```
#correct o/p of seven numbers – range starts at 1
```

```
# maxNum never outputs
```

Using var 'maxNum' as a parameter:
Must be an integer variable

We can output 1 2 3 4 5 6 7 8

```
#increment max_value in the header - tends to be the preferred choice
```

```
for i in range (1, maxNum+1):
```

```
    print (i, end=" ")
```

OR

```
#leave the header and change the print statement
```

```
for i in range (1,maxNum):
```

```
    print (i+1,"",end=" ")
```

o/p index value of a sequence

```
numbers = (1,2,3,4,5)
```

```
length_of_numbers = len(numbers)
```

```
for i in range (length_of_numbers):
```

```
    print ('Index = ', i , " ", 'Value of number = ',numbers[i])
```

```
Index = 0  Value of number = 1
```

```
Index = 1  Value of number = 2
```

```
Index = 2  Value of number = 3
```

```
Index = 3  Value of number = 4
```

```
Index = 4  Value of number = 5
```

- Loops starts with i=0 (the default) unless over-ridden

for loop: in() vs in range()

for <COUNTER> in (sequence):

<STATEMENTS>

for i in (1,2,3): #can't have 'for i in (3)'

print (i)

1 2 3

(sequence) - as many items as needed, comma separated. Sequence has three items so the loop iterates three times - o/p starts at 1

for <COUNTER> in range (sequence) :

<STATEMENTS>

for i in range (9): #max_value

print (i)

0 1 2 3 4 5 6 7 8

in range (max_value):

Start value defaults to 0, end value always end_value -1; e.g. range(9) processes a range of 9 values: 0 to 8

for i in range(3,7): #min_value, max_value

print(i)

3 4 5 6

#start value is now 3, not 0

in range' (min_value, max_value):

Override the default start value, end value always end_value-1; range(3,7) is range of four values start 3, end 6

for loop: summary

Information only

- for always executes the specified number of times
- Never change the counter variable within the loop
- indexing default start = 0
 - May need a workaround to o/p a full sequence
- Can be 'nested'
 - Have a loop within a loop – later lecture

for loop: summary

Information only

for counter in range (sequence): statements

- 'for' is a reserved word
 - can only use it when building a for loop
- counter always an integer
 - usually designated as i, j, k - but can be any name
 - must not be changed in the loop – done automatically
- 'in' is a reserved word to check for an item in the sequence
- 'range' returns a sequence of numbers
 - Start at 0 (default) increment by 1 (default)
- sequence is an 'iterable'
 - for loop always works in combination with an iterable
 - loop iterates over an item ('index', 'element') of the sequence

for loop: summary

Information only

- ‘Definitive iteration’
 - Know how many times to loop
- Reduces code complexity
 - Specified block of code executes a fixed number of times efficiently
- Iterate through sequences
 - Can work with large volumes

for loop: extensions

Information only

- Loop can contain
 - for- else
 - not the same as if-else

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```

- break
 - prematurely exits the loop
- continue
 - skips an item and continues in the loop

Don't use for-else, for-break, for-continue in COMP101

Good explanation of for-else-break at:
https://book.pythontips.com/en/latest/for_-_else.html

When using a for loop to search for an item the loop ends either when item found or item not found. When found use break to exit the loop; When not found use else to explain item is not found.

For loop: range()

- <https://pynative.com/python-range-function/>